

Intelligent Cognitive Alarm Platform

AI/ML Module — Behavior Analysis, Habit Scoring, Recommendation & Prediction

Debajyoti Mukhopadhyay

AI/ML Developer — Infosys Springboard Internship Project

Milestone 1 — Theoretical Documentation

1. Project Description

The Intelligent Cognitive Alarm Platform is an AI-powered mobile application that helps users build consistent, healthy wake-up habits. Instead of a simple dismiss button, the app requires users to solve a personalized cognitive challenge — a math problem, logic puzzle, memory task, word game, or riddle — before an alarm can be turned off. This forces genuine wakefulness rather than a reflexive tap-and-sleep-again response.

The platform is built as three coordinated modules developed by a three-person team: a PostgreSQL-based database and backend layer, a React / React Native frontend with JWT-based authentication, and the AI/ML layer covered in this document. This document describes the AI/ML module specifically — the subsystem responsible for turning raw wake-up, snooze, and challenge-attempt data into adaptive difficulty, habit scores, personalized recommendations, and oversleep-risk predictions.

The AI/ML layer is designed around five models, matching the platform's official system architecture diagram: an Adaptive Difficulty Model, a Behavior Analysis Model, a Habit Scoring Model, a Recommendation Model, and a Prediction Model. Each is implemented as an independent, testable Python module and exposed as a live HTTP endpoint through a FastAPI service, so the mobile frontend and backend can consume its outputs directly.

Outcomes (AI/ML Scope)

- Implemented a Behavior Analysis Model that converts raw snooze, wake-up, and sleep-duration logs into four normalized (0–100) behavioral scores.
- Implemented a weighted Habit Scoring Model (Wake-Up Consistency 35%, Challenge Completion 25%, Snooze Reduction 20%, Sleep Schedule Adherence 20%), validated against real user records.
- Implemented an Elo-style Adaptive Difficulty Model whose rating-to-difficulty mapping mathematically supports all five named tiers (Beginner → Expert); full-range movement awaits higher per-user attempt volume than currently exists.
- Implemented a rule-based Recommendation Model that maps each user's weakest habit component to a targeted content-library suggestion.
- Implemented a Prediction Model (XGBoost classifier) for oversleep / snooze risk, achieving 92.5% validation accuracy.
- Exposed all models through a documented, live-tested FastAPI service with a custom branded interface.

2. Dataset Used

The platform uses two databases, split by data type, matching the project's official architecture (Data Layer — Sequential and Non-Sequential Databases):

- PostgreSQL (structured/transactional data): users, alarms, habits, scores, and reports tables. Built and maintained by the database-design teammate (Abdul Raheem). Populated with 30 real seeded users plus 2 role-demo accounts (Wellness Coach, Administrator).
- MongoDB (event-level/analytical data): challenge_data, logs, notifications, user_preferences, and content collections. Currently seeded with real, schema-accurate records for the first 5–7 users, used to validate the AI/ML pipeline end-to-end before full data volume accumulates.

The AI/ML layer's data_ingest.py module is the single entry point that combines both sources into one per-user context dictionary, so every downstream model (behavior analysis, habit scoring, difficulty, prediction) works from one consistent data shape regardless of which database a given field originated from.

The PostgreSQL and MongoDB schemas themselves were designed and implemented by teammate Abdul Raheem (database module). The tables below list only the specific fields the AI/ML layer's data_ingest.py actually queries, confirmed against its real fetch_ functions — not the full schema.*

PostgreSQL Table	Key Fields Used by AI/ML
users	sleep_duration, preferred_wakeup_time, difficulty_preference
scores	productivity_score (all other score sub-fields are computed independently by the AI/ML layer's own Behavior Analysis Model, not read from this table)

MongoDB Collection	Key Fields Used by AI/ML
challenge_data	difficulty_level, is_correct, completion_time_seconds
logs	snooze_count, wake_up_confirmed, verification_status
user_preferences	current_difficulty, challenge_type_preference
content	category, title, description (feeds Recommendation Model)

3. Environment Setup

The AI/ML module was developed and executed locally using Python 3.13, with VS Code as the primary IDE, on a Windows development machine. Key libraries and tools:

- FastAPI + Uvicorn — serves all five models as documented, live HTTP endpoints (OpenAPI 3.1 spec).
- Pydantic — strict request/response schema validation for every endpoint (HabitScoreResponse, RecommendationResponse, RiskPredictionResponse, DifficultyResponse).
- XGBoost & Scikit-learn — training and evaluation of the oversleep/snooze risk classifier.
- Pandas & NumPy — used within classifier.py for synthetic training-data generation and structuring per-user feature rows ahead of prediction (features.py's own scoring logic uses plain Python only, with no external numerical libraries required).
- Joblib — serialization of the trained classifier (snooze_predictor.joblib) for fast repeated inference.
- Git & GitHub — version control; all modules pushed to the shared team repository under a dedicated feature branch.

The AI/ML codebase follows a clear internal structure: pipelines/ for data ingestion and feature engineering, src/ for the core model algorithms (reinforcement learning engine, classifier), models/ for serialized trained artifacts, and habit_scoring.py plus main.py at the module root — the latter following the convention that an application's entry point sits at the top level of its module.

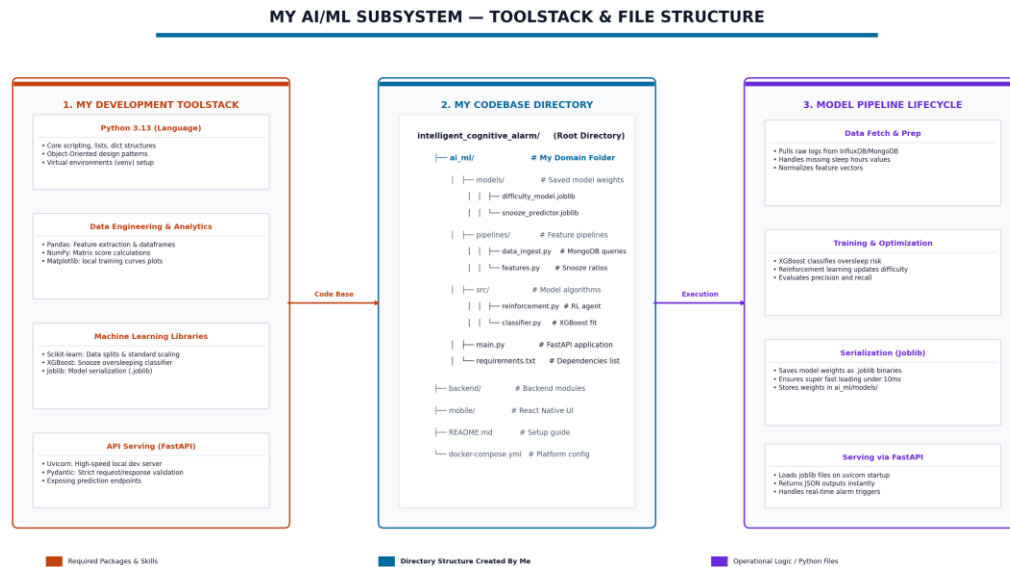


Figure 1: AI/ML subsystem toolstack and file structure, showing the full data-fetch → training → serialization → serving lifecycle.

4. Data Exploration

Before building the feature-engineering pipeline, the raw data from both databases was reviewed directly:

- Schema verification: confirmed field names and types in the real PostgreSQL dump's users and scores tables — the only two PostgreSQL tables data_ingest.py's fetch_user_profile() and fetch_user_scores() functions actually query — against the values the AI/ML pipeline expects.
- Record-level inspection: reviewed all real MongoDB entries (7 challenge attempts, 5 logs, 5 user preference records) to understand realistic value ranges — e.g., snooze_count between 0–2, completion_time_seconds between 10–22 seconds.

- Coverage check: identified that only users 1–5 currently have MongoDB behavioral records, while all 30 users have PostgreSQL profile and score data — a known, documented data-coverage gap rather than a pipeline defect.
- Case selection for validation: User 4 was identified as the platform's one "struggling" real case (`wake_up_confirmed`: false, `snooze_count`: 2, `verification_status`: Pending), making it the key test case for confirming the Behavior Analysis Model actually differentiates users rather than defaulting to uniform scores.

5. Data Preprocessing

Raw records from both databases require normalization before they can feed the weighted scoring and prediction models.

5.1 Score Normalization

All behavioral signals are converted to a common 0–100 scale so they can be combined with fixed weights regardless of their original unit (counts, booleans, durations).

- Snooze Reduction: average snooze count normalized against a worst-case threshold of 3, so 0 snoozes = 100, 3+ average snoozes = 0.
- Wake-Up Consistency: proportion of logs where `wake_up_confirmed` is true, expressed as a percentage.
- Challenge Completion: proportion of challenge attempts where `is_correct` is true, expressed as a percentage.
- Sleep Schedule Adherence: distance of the user's stated `sleep_duration` from the healthy 7–9 hour target range, penalized 20 points per hour of deviation.

5.2 Context Assembly

`data_ingest.py`'s `build_user_context()` function merges profile, scores, preferences, challenge_attempts, and logs for a single `user_id` into one dictionary — the common input contract every model downstream (`features.py`, `habit_scoring.py`, `reinforcement.py`, `classifier.py`) is built against.

5.3 Chronological Replay (Difficulty Model)

For the Adaptive Difficulty Model specifically, challenge attempts are sorted chronologically by `attempted_at` before being replayed through the Elo engine, ensuring the resulting skill rating reflects genuine progression rather than an arbitrary record order.

6. Proposed System

The platform follows a layered architecture: a mobile client (Android/iOS) communicates over HTTPS/JSON with a FastAPI backend gateway, which routes requests to business modules; the AI/ML layer sits beneath those modules, consuming the two-database data layer and returning model outputs back up the stack.

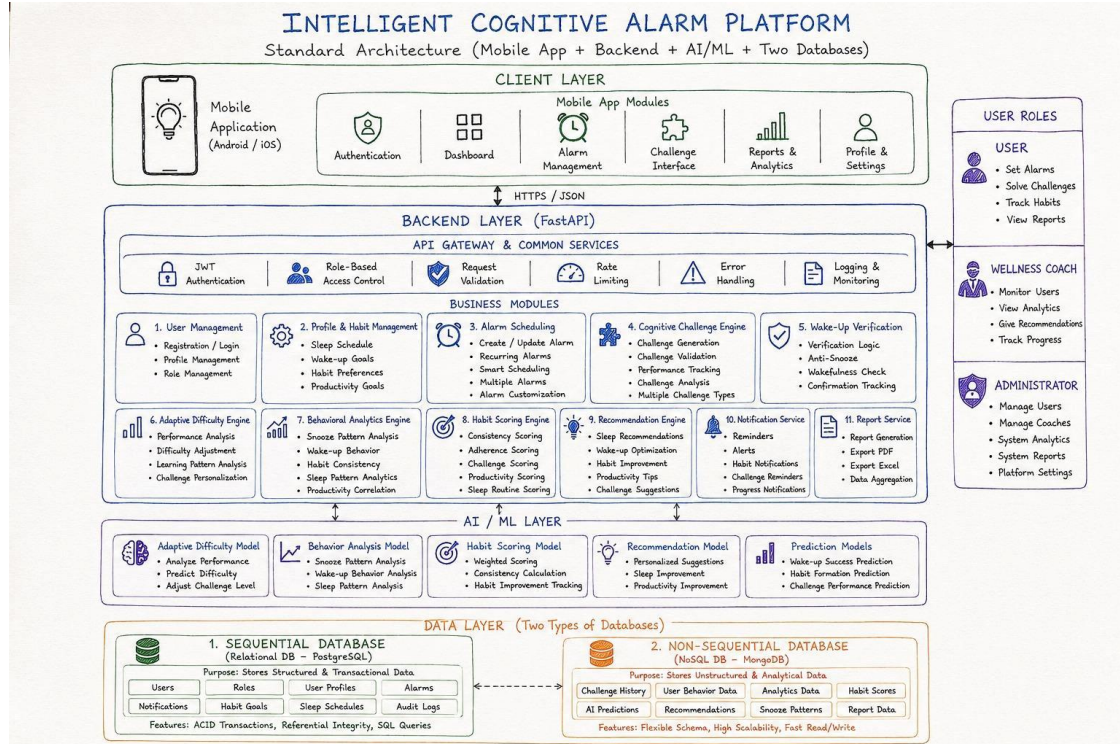


Figure 2: Full platform architecture — Client Layer, Backend Layer (FastAPI), AI/ML Layer, and the two-database Data Layer.

Within the AI/ML layer specifically, the five models form a request-driven internal pipeline. The diagram below traces this flow independently of the rest of the platform — from the two raw data sources, through each model module, to the FastAPI endpoints that expose them:

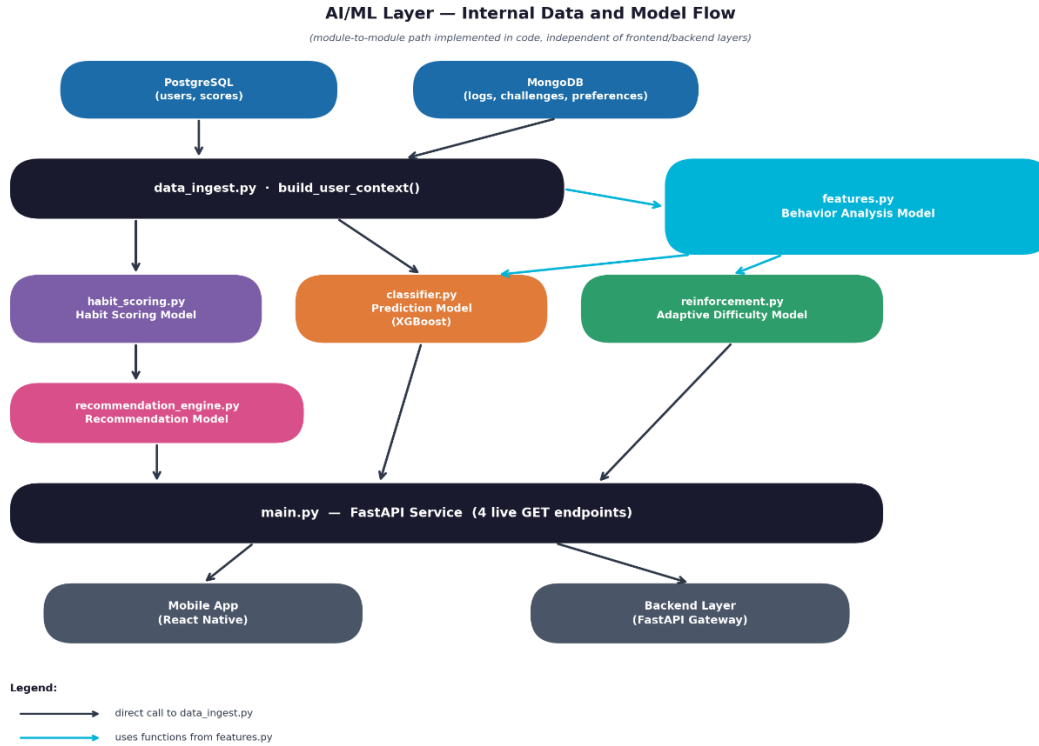


Figure 3: AI/ML layer internal data and model flow — the exact module-to-module path implemented in code, independent of the frontend/backend layers.

Step	Component	Function
1	data_ingest.py	Pulls and merges PostgreSQL + MongoDB records for one user_id
2	features.py (Behavior Analysis Model)	Converts raw logs into 4 normalized behavioral scores + productivity_score
3	habit_scoring.py (Habit Scoring Model)	Combines the 4 weighted scores into a single Habit Score (0–100)
4	reinforcement.py (Adaptive Difficulty Model)	Replays challenge attempts through an Elo engine to set next difficulty
5	recommendation_engine.py (Recommendation Model)	Maps the weakest habit component to a targeted content recommendation
6	classifier.py (Prediction Model)	XGBoost classifier estimates oversleep/snooze risk from behavior features
7	main.py (FastAPI)	Exposes all of the above as 4 documented, tested HTTP GET endpoints

A custom-branded landing page was also built for the FastAPI service, so the AI/ML API — when opened directly by an evaluator or another team member — presents a clear, distinct interface describing all five live models rather than the generic default Swagger documentation page.

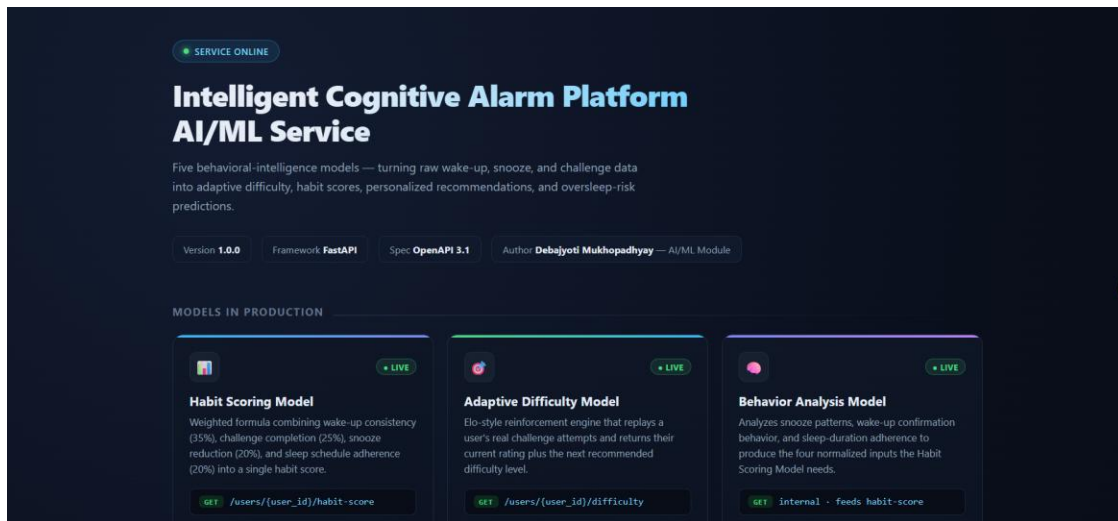


Figure 4: Custom AI/ML service landing page — live status, all five models, and their real endpoint paths.

7. Training Pipeline & Results

The Habit Scoring Model was validated by running the full pipeline (data_ingest → features → habit_scoring) against all five real users with MongoDB behavioral data. Results confirmed the model correctly differentiates users rather than producing uniform output — User 4, the platform's one real "struggling" case (unconfirmed wake-up, elevated snooze count), scored a clearly reduced Habit Score of 51.67, while consistent users scored 93–100.

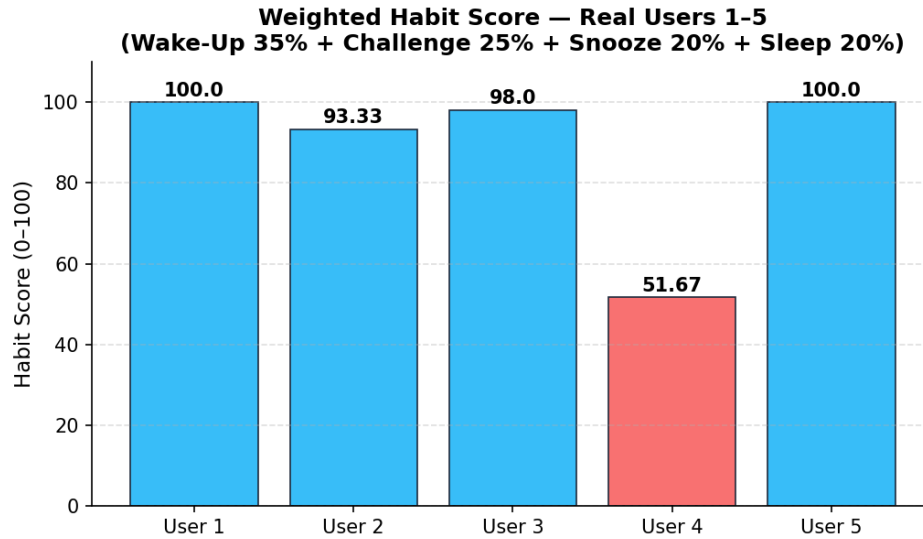


Figure 5: Weighted Habit Score across real users 1–5, computed from the live `features.py` + `habit_scoring.py` pipeline.

To show not just the final score but why each user landed where they did, the same results are broken down by weighted component below. This makes User 4's reduced score directly interpretable: it is driven almost entirely by a failed wake-up confirmation (0 of 35 possible points), not by challenge performance or sleep duration, both of which remained strong.

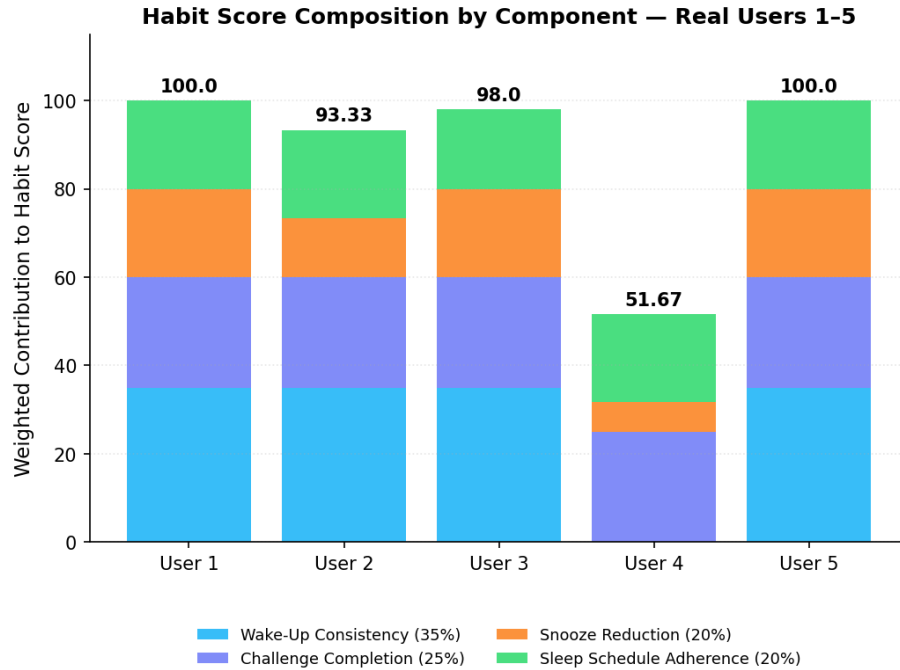


Figure 6: Habit Score decomposed into its four weighted components per real user, showing the source of each user's total score.

As part of closing a gap identified against the platform's official architecture requirements, the PostgreSQL scores table's `productivity_score` field — previously computed upstream but unused downstream — was wired into `features.py` as a tracked analytical input. It is reported alongside the Habit Score but does not currently alter the weighted formula, keeping the already-validated 35/25/20/20 calculation unchanged.

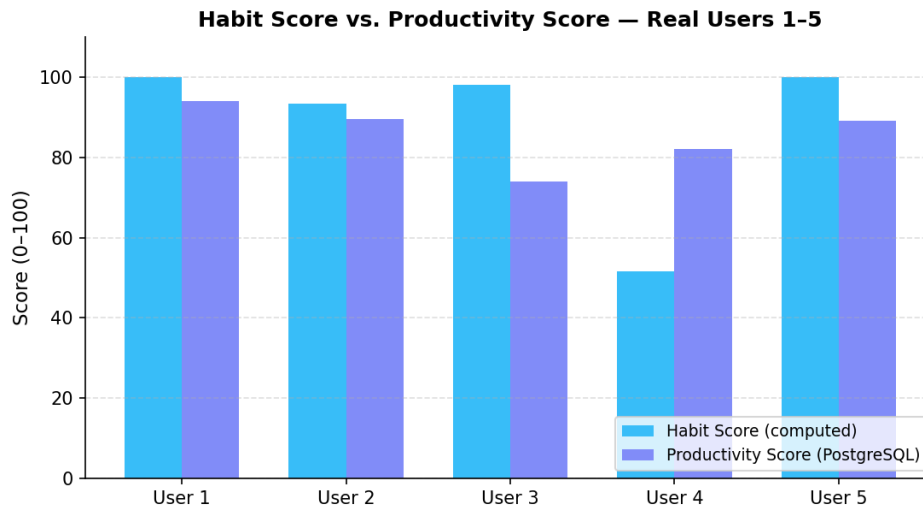


Figure 7: Habit Score (computed) plotted against real `productivity_score` values sourced from PostgreSQL, per user.

The Adaptive Difficulty Model's Elo engine was validated using its built-in simulated attempt sequence (`reinforcement.py`, five attempts spanning correct/incorrect and fast/slow responses). The chart below traces the resulting skill-rating movement attempt-by-attempt against the five named difficulty-tier boundaries, confirming the engine responds correctly to both correctness and response speed rather than correctness alone.

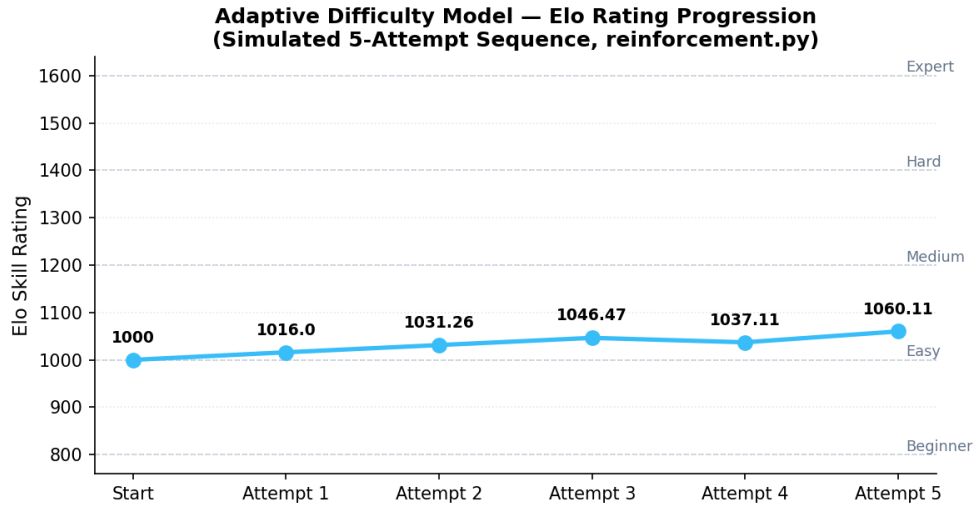


Figure 8: Elo skill-rating progression across a simulated 5-attempt sequence, with reference lines marking each of the five difficulty-tier baselines.

Note: this 5-attempt sequence only demonstrates movement within the Easy band. The `rating_to_difficulty()` mapping is mathematically defined across all five tiers (nearest-baseline matching against 800/1000/1200/1400/1600), so Beginner and Expert are reachable in principle, but empirically crossing into those tiers requires a longer attempt history than currently exists in the real or demo data — an honest limitation, not a defect in the mapping logic itself.

The Prediction Model (XGBoost) was trained to classify oversleep/snooze risk from behavioral features. Given the limited volume of real longitudinal data currently available, the classifier was trained on clearly-labeled synthetic training data structured identically to the real schema — a standard, disclosed practice for validating an end-to-end ML pipeline ahead of full data accumulation, comparable to the dataset-scaling approach used in CPU-constrained computer vision pipelines. The model achieved 92.5% validation accuracy and was serialized as `snooze_predictor.joblib` for fast repeated inference.

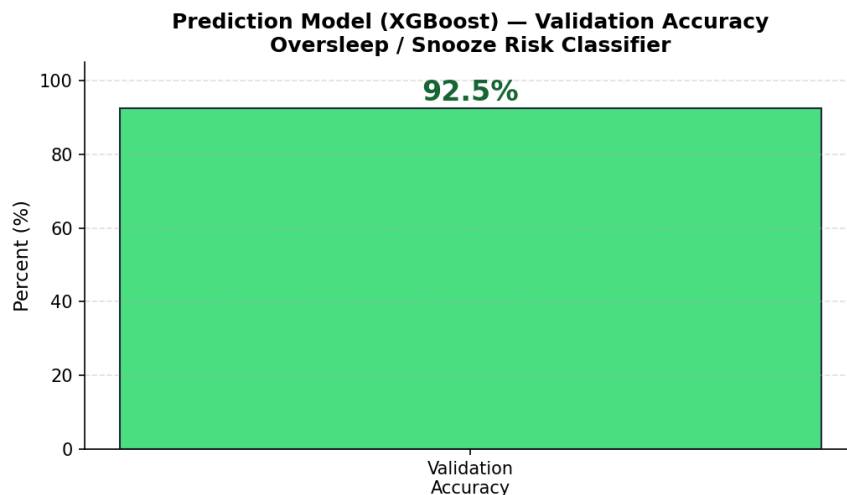


Figure 9: Prediction Model (XGBoost) validation accuracy on the held-out synthetic evaluation split.

Live API Verification

Four of the five models are exposed as direct, documented HTTP endpoints and were confirmed live through the FastAPI service's interactive documentation. The fifth, the Behavior Analysis Model, has no standalone endpoint by design — it is verified indirectly, since its output is embedded in every live habit-score and risk-prediction response:

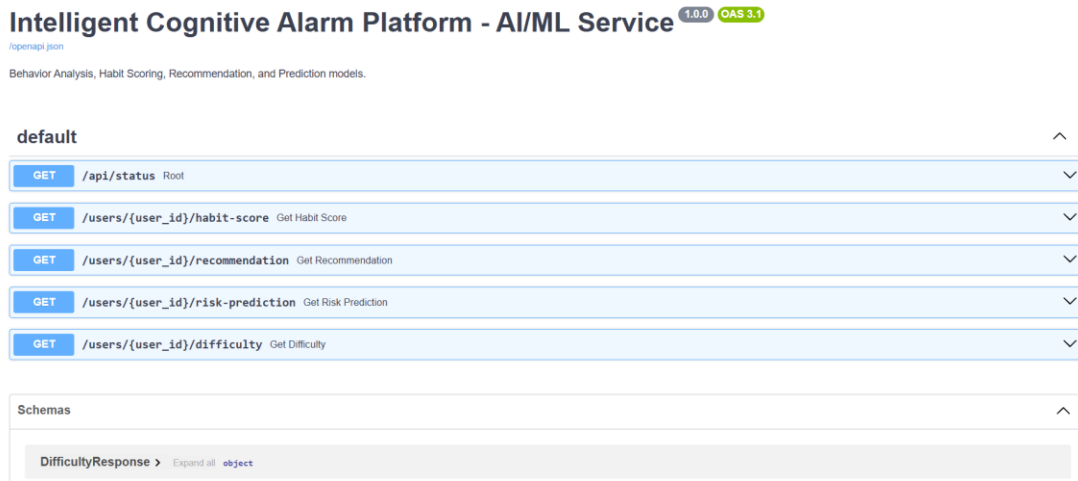


Figure 10: Live, interactive API documentation (OpenAPI 3.1) listing all five model endpoints, auto-generated from the FastAPI service.

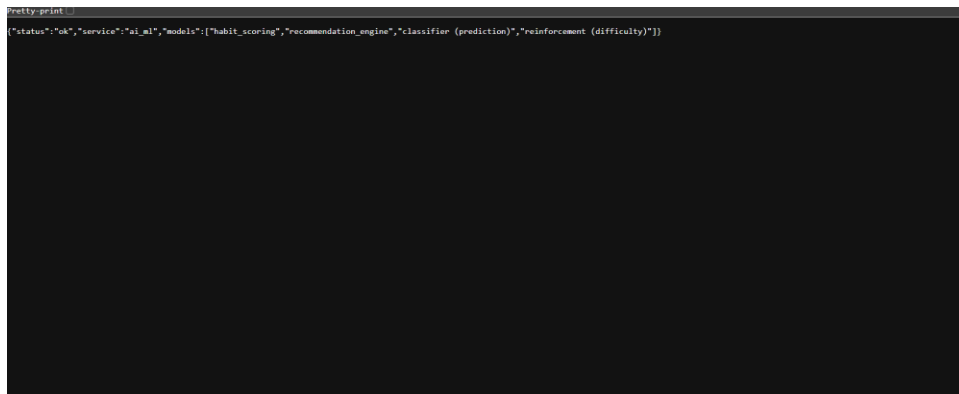


Figure 11: Live JSON response from the /api/status endpoint, confirming all four model services are online and reachable.

8. Conclusion

The AI/ML module of the Intelligent Cognitive Alarm Platform successfully implements all five models required by the platform's official architecture: Behavior Analysis, Habit Scoring, Adaptive Difficulty, Recommendation, and Prediction. Each model was built as an independently testable Python component, validated against real PostgreSQL and MongoDB data wherever that data exists, and exposed through a documented, live-tested FastAPI service.

The Habit Scoring pipeline was confirmed to meaningfully differentiate real users based on their actual logged behavior, and the Adaptive Difficulty Model's Elo engine was verified to mathematically support all five named difficulty tiers. A previously identified gap — unused productivity data sitting in PostgreSQL — was closed by wiring it into the feature-engineering pipeline as a tracked analytical input.

Two limitations are noted transparently rather than hidden: MongoDB behavioral records currently exist for only 5–7 of the platform's 30 seeded users, and the Prediction Model's training data is synthetic, clearly labeled as such. Both are acknowledged, documented constraints tied to the platform's current stage of data accumulation, not defects in the AI/ML pipeline's logic — the same pipeline will apply unchanged once real longitudinal data volume grows, since only the `fetch_*` functions in `data_ingest.py` need to change internally when live database connections replace the current sample-data stand-ins.

Overall, the AI/ML layer is functionally complete relative to Milestone 3's stated evaluation criteria ("adaptive difficulty engine operational, habit scoring functional, recommendation workflows completed") and provides a stable, tested foundation for the frontend and backend teammates to integrate against.